

PRACTICAL FILE

Keshav Mahavidyalaya
University of Delhi

Paper: Blockchain and Its Applications
Paper code: 2343010030

Name: Upasna Kukreti
Course: BSc (H) Computer Science
Roll No.: 2216126
Semester: 8th

Q1: Using SHA256, obtain the message digest of string **"Blockchain Developer"**.

```
#include <iostream>
#include <openssl/sha.h>
#include <iomanip>
#include <sstream>
using namespace std;
int main() {
    string message = " Blockchain Developer"; // leading space included
    unsigned char hash[SHA256_DIGEST_LENGTH];
    SHA256((unsigned char*)message.c_str(), message.length(), hash);
    cout << "Message: \"" << message << "\"" << endl;
    cout << "SHA-256 Digest: ";
    for (int i = 0; i < SHA256_DIGEST_LENGTH; i++) {
        cout << hex << setw(2) << setfill('0') << (int)hash[i];
    }
    cout << endl;
    return 0;
}
```

```
upasna_kukreti@Pond:~$ g++ bc_prac1.cpp -o prac1 -lcrypto
upasna_kukreti@Pond:~$ ./prac1
Message: " Blockchain Developer"
SHA-256 Digest: 688102c2efb8169cb6fce450ab7459b98da583171e48b1a4e3088447f1a0cb51
```

Q2 Write a program to encrypt and decrypt the message **"Hello World"** using **SHA256**.

```
#include <iostream>
#include <openssl/sha.h>
#include <iomanip>
using namespace std;
int main() {
    string msg = "Hello World";
    unsigned char hash[SHA256_DIGEST_LENGTH];
    SHA256((unsigned char*)msg.c_str(), msg.length(), hash);
    cout << "Message : " << msg << endl;
    cout << "SHA256 : ";
    for (int i = 0; i < SHA256_DIGEST_LENGTH; i++)
        cout << hex << setw(2) << setfill('0') << (int)hash[i];
    cout << endl;
    cout << "Decryption : Not possible (SHA256 is one-way)" << endl;
    return 0;
}
```

```
upasna_kukreti@Pond:~$ nano bc_prac2progl.cpp
upasna_kukreti@Pond:~$ g++ bc_prac2progl.cpp -o progl -lcrypto
upasna_kukreti@Pond:~$ ./progl
Message : Hello World
SHA256 : a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
Decryption : Not possible (SHA256 is one-way)
upasna_kukreti@Pond:~$ |
```

Q3 Implement **RSA cryptographic algorithm**.

p = 17

q = 11

e = 11

M = 12

n = p * q

phi = (p - 1) * (q - 1)

```
def mod_inverse(e, phi):  
    for d in range(1, phi):  
        if (e * d) % phi == 1:  
            return d
```

d = mod_inverse(e, phi)

C = pow(M, e, n)

M_decrypted = pow(C, d, n)

```
print("-----RSA-----\n\n")  
print("GIVEN:-")  
print("Value of M:", M)  
print("Value of e:", e)  
print("Value of p:", p)  
print("Value of q:", q)  
print("-----\n")  
print("Value of n:", n)  
print("Value of phi(n):", phi)  
print("Private key d:", d)  
print("Ciphertext:", C)  
print("Decrypted message:", M_decrypted)
```

```
*** -----RSA-----  
  
GIVEN:-  
Value of M: 12  
Value of e: 11  
Value of p: 17  
Value of q: 11  
-----  
  
Value of n: 187  
Value of phi(n): 160  
Private key d: 131  
Ciphertext: 23  
Decrypted message: 12
```

Q4 Create a simple blockchain using **Proof of Work (PoW)**.

```
import hashlib
```

```
class Block:
```

```
    def __init__(self, index, data, previous_hash):
```

```
        self.index = index
```

```
        self.data = data
```

```
        self.previous_hash = previous_hash
```

```
        self.nonce = 0
```

```
        self.hash = self.mine_block()
```

```
    def calculate_hash(self):
```

```
        value = str(self.index) + self.data + self.previous_hash + str(self.nonce)
```

```
        return hashlib.sha256(value.encode()).hexdigest()
```

```
    def mine_block(self):
```

```
        hash_value = self.calculate_hash()
```

```
        while not hash_value.startswith("00"):
```

```
            self.nonce += 1
```

```
            hash_value = self.calculate_hash()
```

```
        return hash_value
```

```
class Blockchain:
```

```
    def __init__(self):
```

```
        self.chain = []
```

```
    def add_block(self, data):
```

```
        if len(self.chain) == 0:
```

```
            previous_hash = "0"
```

```
        else:
```

```
            previous_hash = self.chain[-1].hash
```

```
        block = Block(len(self.chain), data, previous_hash)
```

```
        self.chain.append(block)
```

```
    def print_chain(self):
```

```
        for block in self.chain:
```

```
            print("Index:", block.index)
```

```
            print("Data:", block.data)
```

```
            print("Hash:", block.hash)
```

```
            print("Previous Hash:", block.previous_hash)
```

```
print("Nonce:", block.nonce)
print("-" * 40)
```

```
my_blockchain = Blockchain()
```

```
my_blockchain.add_block("A -> B")
my_blockchain.add_block("B -> C")
my_blockchain.add_block("C -> D")
```

```
my_blockchain.print_chain()
```

```
***  Index: 0
      Data: A -> B
      Hash: 00978c4a6ffb0cdd67a84be6048bda8b5b8b7ef4420f93b917382fafdafcb978
      Previous Hash: 0
      Nonce: 63
      -----
      Index: 1
      Data: B -> C
      Hash: 006b2568265d557315cdb46a193e7445adc9fe3924b0cf33474fdc006facb763
      Previous Hash: 00978c4a6ffb0cdd67a84be6048bda8b5b8b7ef4420f93b917382fafdafcb978
      Nonce: 109
      -----
      Index: 2
      Data: C -> D
      Hash: 00b7aaf237e106531e1e18e95ade127a7993c1dfefd164a7f69dd2abdd282d68
      Previous Hash: 006b2568265d557315cdb46a193e7445adc9fe3924b0cf33474fdc006facb763
      Nonce: 161
      -----
```

Q5 Demonstrate sending of a **digitally signed document**.

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
```

```
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
```

```
public_key = private_key.public_key()
```

```
document = b"This is a digitally signed document"
```

```
print("Original Document:", document.decode())
```

```
signature = private_key.sign(
    document,
    padding.PKCS1v15(),
    hashes.SHA256()
)
```

```
print("Document signed successfully")
```

```
public_key.verify(
    signature,
    document,
    padding.PKCS1v15(),
    hashes.SHA256()
)
```

```
print("Signature verified successfully")
```

```
Original Document: This is a digitally signed document
Document signed successfully
Signature verified successfully
```

Q6 Create a blockchain block containing **block hash, transaction history, time of creation.**

```
import hashlib
```

```
import datetime
```

```
class Block:
```

```
    def __init__(self, index, timestamp, data, previous_hash):
```

```
        self.index = index
```

```
        self.timestamp = timestamp
```

```
        self.data = data
```

```
        self.previous_hash = previous_hash
```

```
        self.hash = self.calculate_hash()
```

```
    def calculate_hash(self):
```

```
        block_string = str(self.index) + str(self.timestamp) + str(self.data) + str(self.previous_hash)
```

```
        return hashlib.sha256(block_string.encode()).hexdigest()
```

```
class Blockchain:
```

```
    def __init__(self):
```

```
        self.chain = [self.create_genesis_block()]
```

```
    def create_genesis_block(self):
```

```
        return Block(0, datetime.datetime.now(), "Genesis Block", "0")
```

```
    @property
```

```
    def last_block(self):
```

```
        return self.chain[-1]
```

```
    def add_block(self, new_block):
```

```
        new_block.previous_hash = self.last_block.hash
```

```
        new_block.hash = new_block.calculate_hash()
```

```
        self.chain.append(new_block)
```

```
my_blockchain = Blockchain()
```

```
print("Blockchain created with Genesis Block.")
```

```
print("Adding blocks...")
```

```
my_blockchain.add_block(
```

```
    Block(1, datetime.datetime.now(), {
```

```
        "sender": "Alice",
```

```
        "recipient": "Bob",
```

```
        "amount": 10.0
```

```
    }, ""))
```

```
)
```



```

print("Block 1 added.")
my_blockchain.add_block(
    Block(2, datetime.datetime.now(), {
        "sender": "Bob",
        "recipient": "Charlie",
        "amount": 5.0
    }, "")
)
print("Block 2 added.")
my_blockchain.add_block(
    Block(3, datetime.datetime.now(), {
        "sender": "Charlie",
        "recipient": "Alice",
        "amount": 3.5
    }, "")
)
print("Block 3 added.")

print("\n--- Blockchain Content ---")
for block in my_blockchain.chain:
    print(f"\nBlock Index: {block.index}")
    print(f"Timestamp: {block.timestamp}")
    print(f>Data (Transactions): {block.data}")
    print(f"Previous Hash: {block.previous_hash}")
    print(f"Block Hash: {block.hash}")

```

```

Blockchain created with Genesis Block.
Adding blocks...
Block 1 added.
Block 2 added.
Block 3 added.

--- Blockchain Content ---

Block Index: 0
Timestamp: 2026-04-12 08:11:20.097571
Data (Transactions): Genesis Block
Previous Hash: 0
Block Hash: 3314e1b1a409fd7a6e83221d173711e9f0c2ba99a1f9d10b81a8cf85d5b953dc

Block Index: 1
Timestamp: 2026-04-12 08:11:20.097844
Data (Transactions): {'sender': 'Alice', 'recipient': 'Bob', 'amount': 10.0}
Previous Hash: 3314e1b1a409fd7a6e83221d173711e9f0c2ba99a1f9d10b81a8cf85d5b953dc
Block Hash: 2737d58b97d441783ca6b2ff5b56ad57387ef9aeb4f4459907dc5a17aafed32d

Block Index: 2
Timestamp: 2026-04-12 08:11:20.098001
Data (Transactions): {'sender': 'Bob', 'recipient': 'Charlie', 'amount': 5.0}
Previous Hash: 2737d58b97d441783ca6b2ff5b56ad57387ef9aeb4f4459907dc5a17aafed32d
Block Hash: 27cf5c29ce3002f349424c6bbe3c228a4a0aa0af12024996ea4b65a9e86f6fe9

Block Index: 3
Timestamp: 2026-04-12 08:11:20.098139
Data (Transactions): {'sender': 'Charlie', 'recipient': 'Alice', 'amount': 3.5}
Previous Hash: 27cf5c29ce3002f349424c6bbe3c228a4a0aa0af12024996ea4b65a9e86f6fe9
Block Hash: 832b22f7fa6f82d1ba403c5cbebe9320af7265b6a70ea03951cd0502062894e0

```

Q7 Create a blockchain having **5 nodes** and print the **hash values of each block**.

```
import hashlib
```

```
import datetime
```

```
class Block:
```

```
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = str(datetime.datetime.now())
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()
```

```
    def calculate_hash(self):
        value = str(self.index) + self.timestamp + self.data + self.previous_hash
        return hashlib.sha256(value.encode()).hexdigest()
```

```
class Blockchain:
```

```
    def __init__(self):
        self.chain = [self.create_genesis_block()]
```

```
    def create_genesis_block(self):
        return Block(0, "Genesis Block", "0")
```

```
    def add_block(self, data):
        last_block = self.chain[-1]
        new_block = Block(len(self.chain), data, last_block.hash)
        self.chain.append(new_block)
```

```
    def print_chain(self):
        for block in self.chain:
            print("Block", block.index)
            print("Data:", block.data)
            print("Hash:", block.hash)
            print("Previous Hash:", block.previous_hash)
            print("-----")
```

```
    def is_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i-1]
```

```

    if current.hash != current.calculate_hash():
        return False

    if current.previous_hash != previous.hash:
        return False

    return True

blockchain = Blockchain()

blockchain.add_block("A -> B")
blockchain.add_block("B -> C")
blockchain.add_block("C -> D")
blockchain.add_block("D -> E")

print("Blockchain with 5 blocks:")
blockchain.print_chain()

```

```

... Blockchain with 5 blocks:
Block 0
Data: Genesis Block
Hash: 2ed9b5a3c94f0a4b68b515e4a30c12ffeb4b0eca07a3195922f67db56d75c600
Previous Hash: 0
-----
Block 1
Data: A -> B
Hash: e13c7456522308d1f7c2ab170301d4bf842b884224b00b474519bd02bb71b94e
Previous Hash: 2ed9b5a3c94f0a4b68b515e4a30c12ffeb4b0eca07a3195922f67db56d75c600
-----
Block 2
Data: B -> C
Hash: 7e0c1767b5d45c9fb91be7c1e6a9b6ce68a1014e039bcf16ca54f2bda4ada937
Previous Hash: e13c7456522308d1f7c2ab170301d4bf842b884224b00b474519bd02bb71b94e
-----
Block 3
Data: C -> D
Hash: bafe319b575481fceb27b5241a1d7a3107edb57ecd0be5e62ce3b6b6a17669b5
Previous Hash: 7e0c1767b5d45c9fb91be7c1e6a9b6ce68a1014e039bcf16ca54f2bda4ada937
-----
Block 4
Data: D -> E
Hash: 00ff0c119ea441260264ec0b8fc17221ead4e8fbb901f30ad50db3bb79532654
Previous Hash: bafe319b575481fceb27b5241a1d7a3107edb57ecd0be5e62ce3b6b6a17669b5
-----

```

Q8 Create a blockchain having **5 nodes** and **check its validity**.

```
import hashlib
```

```
import datetime
```

```
class Block:
```

```
    def __init__(self, index, data, previous_hash):
```

```
        self.index = index
```

```
        self.timestamp = str(datetime.datetime.now())
```

```
        self.data = data
```

```
        self.previous_hash = previous_hash
```

```
        self.hash = self.calculate_hash()
```

```
    def calculate_hash(self):
```

```
        value = str(self.index) + self.timestamp + self.data + self.previous_hash
```

```
        return hashlib.sha256(value.encode()).hexdigest()
```

```
class Blockchain:
```

```
    def __init__(self):
```

```
        self.chain = [self.create_genesis_block()]
```

```
    def create_genesis_block(self):
```

```
        return Block(0, "Genesis Block", "0")
```

```
    def add_block(self, data):
```

```
        last_block = self.chain[-1]
```

```
        new_block = Block(len(self.chain), data, last_block.hash)
```

```
        self.chain.append(new_block)
```

```
    def print_chain(self):
```

```
        for block in self.chain:
```

```
            print("Block", block.index)
```

```
            print("Data:", block.data)
```

```
            print("Hash:", block.hash)
```

```
            print("Previous Hash:", block.previous_hash)
```

```
            print("-----")
```

```
    def is_valid(self):
```

```
        for i in range(1, len(self.chain)):
```

```
            current = self.chain[i]
```

```
            previous = self.chain[i-1]
```

```

        if current.hash != current.calculate_hash():
            return False

        if current.previous_hash != previous.hash:
            return False

    return True

blockchain = Blockchain()

blockchain.add_block("A -> B")
blockchain.add_block("B -> C")
blockchain.add_block("C -> D")
blockchain.add_block("D -> E")

print("Blockchain with 5 blocks:")
blockchain.print_chain()

print("Is blockchain valid?", blockchain.is_valid())

# Tamper with 3rd block
print("\nTampering with Block 3...\n")

blockchain.chain[2].data = "Tampered Data"

print("Is blockchain valid after tampering?", blockchain.is_valid())

print("\nCreating blockchain network with 5 nodes...\n")

nodes = []

for i in range(5):
    node_blockchain = Blockchain()
    node_blockchain.add_block("A -> B")
    node_blockchain.add_block("B -> C")
    node_blockchain.add_block("C -> D")
    node_blockchain.add_block("D -> E")
    nodes.append(node_blockchain)

```

Checking validity of each node

for i in range(5):

print("Node", i+1, "Blockchain valid?", nodes[i].is_valid())

Blockchain with 5 blocks:

Block 0

Data: Genesis Block

Hash: 2ed9b5a3c94f0a4b68b515e4a30c12ffeb4b0eca07a3195922f67db56d75c600

Previous Hash: 0

Block 1

Data: A -> B

Hash: e13c7456522308d1f7c2ab170301d4bf842b884224b00b474519bd02bb71b94e

Previous Hash: 2ed9b5a3c94f0a4b68b515e4a30c12ffeb4b0eca07a3195922f67db56d75c600

Block 2

Data: B -> C

Hash: 7e0c1767b5d45c9fb91be7c1e6a9b6ce68a1014e039bcf16ca54f2bda4ada937

Previous Hash: e13c7456522308d1f7c2ab170301d4bf842b884224b00b474519bd02bb71b94e

Block 3

Data: C -> D

Hash: bafe319b575481fceb27b5241a1d7a3107edb57ecd0be5e62ce3b6b6a17669b5

Previous Hash: 7e0c1767b5d45c9fb91be7c1e6a9b6ce68a1014e039bcf16ca54f2bda4ada937

Block 4

Data: D -> E

Hash: 00ff0c119ea441260264ec0b8fc17221ead4e8fbb901f30ad50db3bb79532654

Previous Hash: bafe319b575481fceb27b5241a1d7a3107edb57ecd0be5e62ce3b6b6a17669b5

Is blockchain valid? True

Tampering with Block 3...

Is blockchain valid after tampering? False

Creating blockchain network with 5 nodes...

Node 1 Blockchain valid? True

Node 2 Blockchain valid? True

Node 3 Blockchain valid? True

Node 4 Blockchain valid? True

Node 5 Blockchain valid? True

Q9 Implement a **smart contract using Solidity programming language.**

```
!pip install py-solc-x web3 eth-tester
```

```
from solcx import install_solc
```

```
install_solc('0.8.0')
```

```
from solcx import compile_source, set_solc_version
```

```
set_solc_version('0.8.0')
```

```
contract_code = ""
```

```
pragma solidity ^0.8.0;
```

```
contract SimpleStorage {
```

```
    uint public data;
```

```
    function setData(uint _data) public {
```

```
        data = _data;
```

```
    }
```

```
    function getData() public view returns(uint) {
```

```
        return data;
```

```
    }
```

```
}
```

```
""
```

```
compiled = compile_source(contract_code)
```

```
compiled
```



```
{
  "contract": "SimpleStorage",
  "abi": [
    {
      "inputs": [],
      "name": "data",
      "outputs": [
        {
          "internalType": "uint256",
          "name": "",
          "type": "uint256"
        }
      ],
      "stateMutability": "view",
      "type": "function"
    },
    {
      "inputs": [],
      "name": "getData",
      "outputs": [
        {
          "internalType": "uint256",
          "name": "",
          "type": "uint256"
        }
      ],
      "stateMutability": "view",
      "type": "function"
    },
    {
      "inputs": [
        {
          "internalType": "uint256",
          "name": "_data",
          "type": "uint256"
        }
      ],
      "name": "setData",
      "outputs": [],
      "stateMutability": "nonpayable",
      "type": "function"
    }
  ],
  "asm": {
    ".code": [
      {
        "begin": 26,
        "end": 224,
        "name": "PUSH",
        "source": 0,
        "value": "80"
      },
      {
        "begin": 26,
        "end": 224,
        "name": "PUSH",
        "source": 0,
        "value": "40"
      },
      {
        "begin": 26,
        "end": 224,
        "name": "MSTORE",
        "source": 0
      },
      {
        "begin": 26,
        "end": 224,
        "name": "CALLVALUE",
        "source": 0
      },
      {
        "begin": 26,
        "end": 224,
        "name": "DUP1",
        "source": 0
      },
      {
        "begin": 26,
        "end": 224,
        "name": "ISZERO",
        "source": 0
      },
      {
        "begin": 26,
        "end": 224,
        "name": "PUSH [tag]",
        "source": 0,
        "value": "1"
      },
      {
        "begin": 26,
        "end": 224,
        "name": "JUMPI",
        "source": 0
      },
      {
        "begin": 26,
        "end": 224,
        "name": "PUSH",
        "source": 0,
        "value": "0"
      },
      {
        "begin": 26,
        "end": 224,
        "name": "DUP1",
        "source": 0
      },
      {
        "begin": 26,
        "end": 224,
        "name": "REVERT",
        "source": 0
      },
      {
        "begin": 26,
        "end": 224,
        "name": "tag",
        "source": 0,
        "value": "1"
      },
      {
        "begin": 26,
        "end": 224,
        "name": "JUMPDEST",
        "source": 0
      },
      {
        "begin": 26,
        "end": 224,
        "name": "POP",
        "source": 0
      },
      {
        "begin": 26,
        "end": 224,
        "name": "PUSH #[$]",
        "source": 0,
        "value": "0000000000000000000000000000000000000000000000000000000000000000"
      },
      {
        "begin": 26,
        "end": 224,
        "name": "DUP1",
        "source": 0
      },
      {
        "begin": 26,
        "end": 224,
        "name": "DUP1",
        "source": 0
      }
    ]
  }
}
```

```
student_contract = ""
```

```
pragma solidity ^0.8.0;
```

```

contract Student {

    string public name;
    uint public age;

    function setStudent(string memory _name, uint _age) public {
        name = _name;
        age = _age;
    }

    function getStudent() public view returns(string memory, uint) {
        return (name, age);
    }
}
'''

```

```

compiled_student = compile_source(student_contract)
compiled_student

```

```

{'<stdin>:Student': {'abi': [{'inputs': [],
  'name': 'age',
  'outputs': [{'internalType': 'uint256', 'name': '', 'type': 'uint256'}],
  'stateMutability': 'view',
  'type': 'function'},
 {'inputs': [],
  'name': 'getStudent',
  'outputs': [{'internalType': 'string', 'name': '', 'type': 'string'},
    {'internalType': 'uint256', 'name': '', 'type': 'uint256'}],
  'stateMutability': 'view',
  'type': 'function'},
 {'inputs': [],
  'name': 'name',
  'outputs': [{'internalType': 'string', 'name': '', 'type': 'string'}],
  'stateMutability': 'view',
  'type': 'function'},
 {'inputs': [{'internalType': 'string', 'name': '_name', 'type': 'string'},
    {'internalType': 'uint256', 'name': '_age', 'type': 'uint256'}],
  'name': 'setStudent',
  'outputs': [],
  'stateMutability': 'nonpayable',
  'type': 'function'}],
  'asm': {'code': [{'begin': 26,
    'end': 310,
    'name': 'PUSH',
    'source': 0,
    'value': '80'},
    {'begin': 26, 'end': 310, 'name': 'PUSH', 'source': 0, 'value': '40'},
    {'begin': 26, 'end': 310, 'name': 'MSTORE', 'source': 0},
    {'begin': 26, 'end': 310, 'name': 'CALLVALUE', 'source': 0},
    {'begin': 26, 'end': 310, 'name': 'DUP1', 'source': 0},
    {'begin': 26, 'end': 310, 'name': 'ISZERO', 'source': 0},
    {'begin': 26, 'end': 310, 'name': 'PUSH [tag]', 'source': 0, 'value': '1'},
    {'begin': 26, 'end': 310, 'name': 'JUMPI', 'source': 0},
    {'begin': 26, 'end': 310, 'name': 'PUSH', 'source': 0, 'value': '0'},
    {'begin': 26, 'end': 310, 'name': 'DUP1', 'source': 0},
    {'begin': 26, 'end': 310, 'name': 'REVERT', 'source': 0},
    {'begin': 26, 'end': 310, 'name': 'tag', 'source': 0, 'value': '1'},
    {'begin': 26, 'end': 310, 'name': 'JUMPDEST', 'source': 0},

```



```

fund_contract = ""
pragma solidity ^0.8.0;

contract FundTransfer {

    address payable public receiver;

    constructor(address payable _receiver) {
        receiver = _receiver;
    }

    function transferFunds() public payable {
        receiver.transfer(msg.value);
    }

}

```

```

compiled_fund = compile_source(fund_contract)
compiled_fund

```

```

... {'<stdin>:FundTransfer': {'abi': [{'inputs': [{'internalType': 'address payable',
    'name': '_receiver',
    'type': 'address'}],
    'stateMutability': 'nonpayable',
    'type': 'constructor'},
  {'inputs': [],
    'name': 'receiver',
    'outputs': [{'internalType': 'address payable',
    'name': '',
    'type': 'address'}],
    'stateMutability': 'view',
    'type': 'function'},
  {'inputs': [],
    'name': 'transferFunds',
    'outputs': [],
    'stateMutability': 'payable',
    'type': 'function'}]},
  'asm': {''.code': [{'begin': 26,
    'end': 263,
    'name': 'PUSH',
    'source': 0,
    'value': '80'},
    {'begin': 26, 'end': 263, 'name': 'PUSH', 'source': 0, 'value': '40'},
    {'begin': 26, 'end': 263, 'name': 'MSTORE', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'CALLVALUE', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'DUP1', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'ISZERO', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'PUSH [tag]', 'source': 0, 'value': '1'},
    {'begin': 93, 'end': 169, 'name': 'JUMPI', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'PUSH', 'source': 0, 'value': '0'},
    {'begin': 93, 'end': 169, 'name': 'DUP1', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'REVERT', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'tag', 'source': 0, 'value': '1'},
    {'begin': 93, 'end': 169, 'name': 'JUMPDEST', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'POP', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'PUSH', 'source': 0, 'value': '40'},
    {'begin': 93, 'end': 169, 'name': 'MLOAD', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'PUSHSIZE', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'CODESIZE', 'source': 0},
    {'begin': 93, 'end': 169, 'name': 'SUB', 'source': 0},

```

Q10 Create a simple **permissioned blockchain** using **Hyperledger Fabric**.

```
./network.sh up
```

```
./network.sh createChannel
```

```
./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl go
```

```
docker ps
```

```
Session Actions Edit View Help
- VERBOSE: false
jq command not found...

Follow the instructions in the Fabric docs to install the prereqs
https://hyperledger-fabric.readthedocs.io/en/latest/prereqs.html
Deploying chaincode failed

(kali㉿kali)-[~/fabric-samples/test-network]
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATE
D             STATUS          PORTS
NAMES
450fe49f575f   hyperledger/fabric-orderer:latest   "orderer"              57 sec
ons ago      Up 57 seconds   0.0.0.0:7050→7050/tcp, [::]:7050→7050/tcp, 0.0.0
.0:7053→7053/tcp, [::]:7053→7053/tcp, 0.0.0.0:9443→9443/tcp, [::]:9443→94
43/tcp      orderer.example.com
f3479cd3e5d3   hyperledger/fabric-peer:latest      "peer node start"      57 sec
ons ago      Up 57 seconds   0.0.0.0:9051→9051/tcp, [::]:9051→9051/tcp, 7051/
tcp, 0.0.0.0:9445→9445/tcp, [::]:9445→9445/tcp
peer0.org2.example.com
4905a7edcaf3   hyperledger/fabric-peer:latest      "peer node start"      57 sec
ons ago      Up 57 seconds   0.0.0.0:7051→7051/tcp, [::]:7051→7051/tcp, 0.0.0
.0:9444→9444/tcp, [::]:9444→9444/tcp
peer0.org1.example.com

(kali㉿kali)-[~/fabric-samples/test-network]
$
```